

Capture The Flag Writeup

Titisan Ma'am Farah



Team Members:

Richie Frederico Wong	001202400052(Captain)
Inria Altje Kalalo	001202400165
Jesynta Ivolaitya Harya	001202400135
Sheryl Sharon Kaparang	001202400158

Digital Forensic Class 1
President University
2025

Table Of Contents

Forensic

Mailer

RGA?

TripleThreat2

Nightmare

lost-d4-Imp0rt4nt-Fil3

goTTa-fix-the-corruptIon

latte

MailMailSnailMail

scout-code2

EzTraverse

Forensic

Mailer

Solved On: 13/04/2026

Solved by: Richie Frederico Wong

Flag Retrieved:

pu-flag{Visa_Payment_Document.zip_customer@fakebank.com_192.168.1.100:
113}

Challenges overview:

Our infrastructure got hijacked somehow, after an employee opened an email, can you find the artifacts? We need to analyze the provided PCAP file to uncover evidence of the attack. Specifically, the objective is to identify the malicious file's name, determine the attacker's email address, and find the destination address where the malicious code sent its request.

Key Findings:

1. Malicious email that the attacker sent to an employee named rbourne.
2. The email contained a file named "Visa_Payment_Document.zip"
3. The email was sent from customer@fakebank.com

Forensic Analysis:

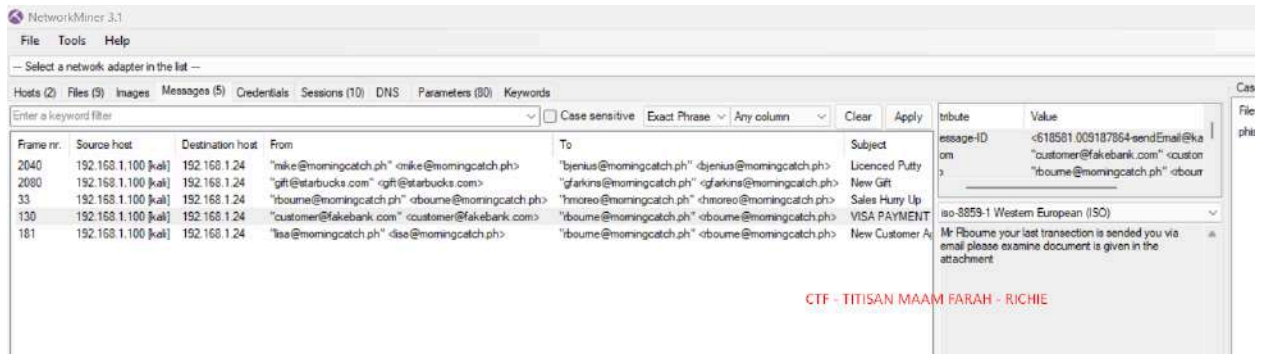
1. The attack was carried out using phishing techniques.
2. The email appeared legitimate because it used a spoofed sender address, "customer@fakebank.com."
3. The attachment included an executable file that downloaded a malicious DLL.
4. The system permitted the execution of rundll32.exe, allowing the malware to run without triggering any warnings.

Tools Used:

1. NetworkMiner

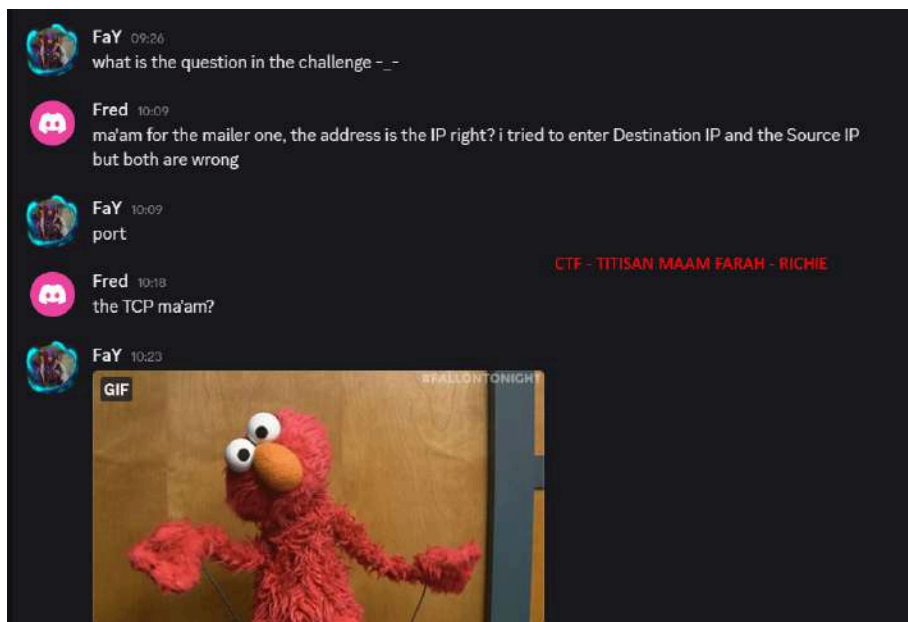
Solving Step-by-step:

1. First, I opened the pcap file with network miner

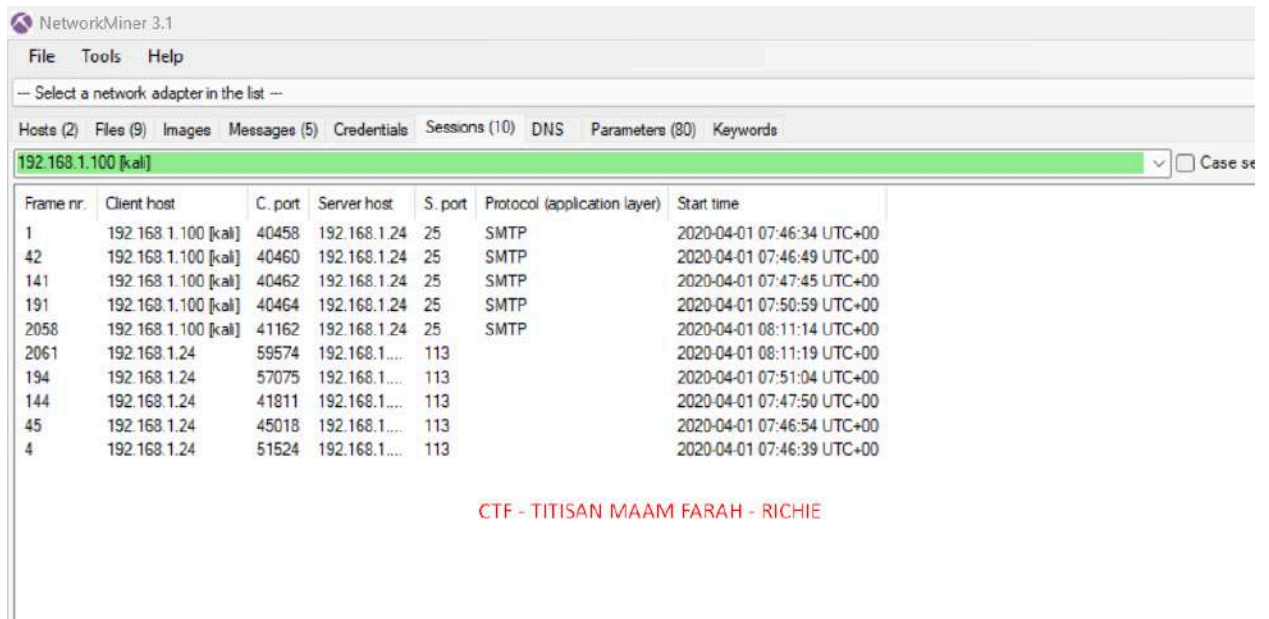


And I saw the message is suspicious because it comes from the suspicious email, and would someone from a company would name their email customer@fakebank, and after this i also saw the files which it the Visa Document Payment and I tried to input it in the Part 1 and it is correct so I move on to the next step which is constructing the flag

2. It's actually kind of difficult because as we can see the address is actually 192.168.1.100 which is the source host, but I got stuck because it's incorrect even i already put all the information to the flag so I asked for a clue



3. When Ma'am Farah told me about port, I go to the Sessions tab



Frame nr.	Client host	C. port	Server host	S. port	Protocol (application layer)	Start time
1	192.168.1.100 [kali]	40458	192.168.1.24	25	SMTP	2020-04-01 07:46:34 UTC+00
42	192.168.1.100 [kali]	40460	192.168.1.24	25	SMTP	2020-04-01 07:46:49 UTC+00
141	192.168.1.100 [kali]	40462	192.168.1.24	25	SMTP	2020-04-01 07:47:45 UTC+00
191	192.168.1.100 [kali]	40464	192.168.1.24	25	SMTP	2020-04-01 07:50:59 UTC+00
2058	192.168.1.100 [kali]	41162	192.168.1.24	25	SMTP	2020-04-01 08:11:14 UTC+00
2061	192.168.1.24	59574	192.168.1.24	113		2020-04-01 08:11:19 UTC+00
194	192.168.1.24	57075	192.168.1.24	113		2020-04-01 07:51:04 UTC+00
144	192.168.1.24	41811	192.168.1.24	113		2020-04-01 07:47:50 UTC+00
45	192.168.1.24	45018	192.168.1.24	113		2020-04-01 07:46:54 UTC+00
4	192.168.1.24	51524	192.168.1.24	113		2020-04-01 07:46:39 UTC+00

CTF - TITISAN MAAM FARAH - RICHIE

As you can see in here we can look at the port which is 113, it is sent by 192.168.1.100 which match the IP from the suspected email and after I got this I construct the flag to become
pu-flag{Visa_Payment_Document.zip_customer@fakebank.com_192.168.1.100:113} and doneeee

RGA?

Solved On: 13-4-2026

Solved by: Inria Altje Kalalo

Flag Retrieved: pu-flag{blu3_ch4nn3l_in_LSB_l34ks_s3cr3ts}

Challenges overview:

An image that looks like a simple red-to-green gradient, but the blue channel which should be completely empty is secretly carrying hidden data in the form of a visible barcode pattern.

Key Findings:

1. Blue channel has pixel values where it should always be 0
2. Isolating the blue channel reveals a clear vertical barcode spanning the full image width
3. The barcode contains the encoded flag

Forensic Analysis:

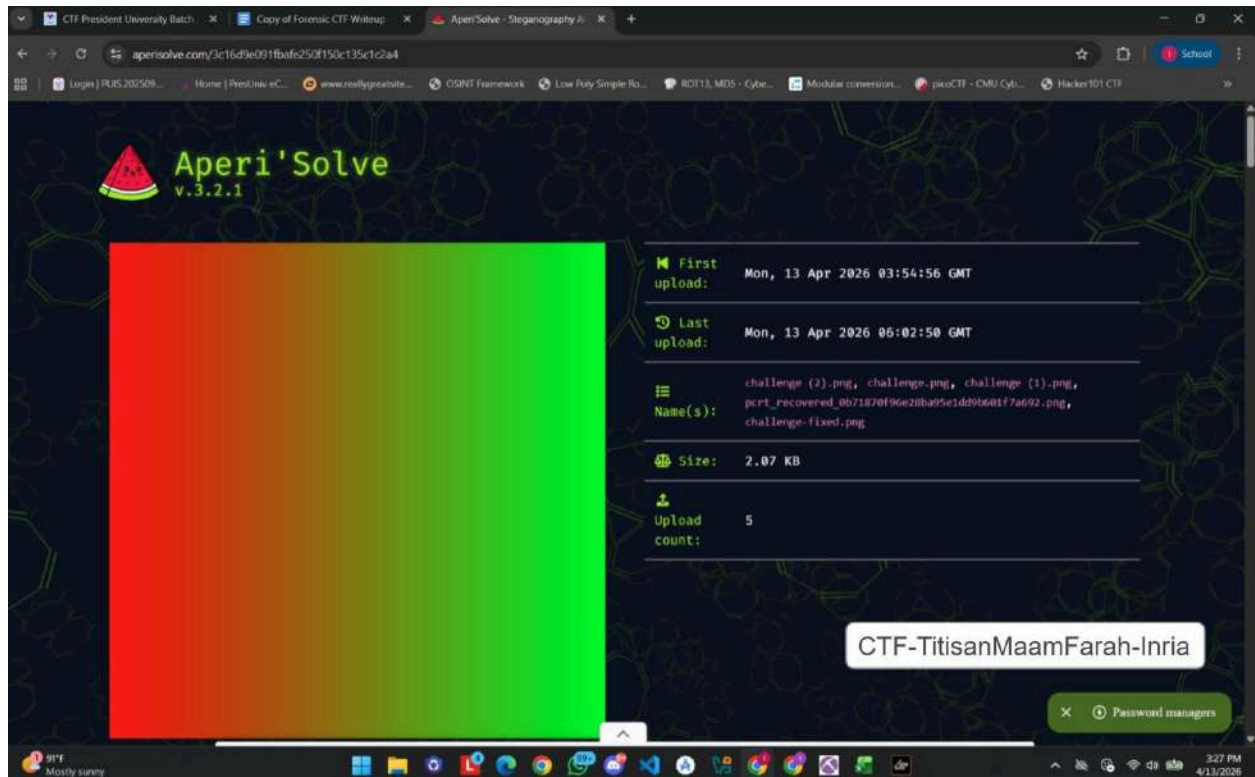
1. Uploaded the image to Aperi'Solve for automated layer and channel analysis
2. Blue channel isolation instantly revealed the hidden barcode pattern
3. Scanned the extracted barcode to retrieve the flag

Tools Used:

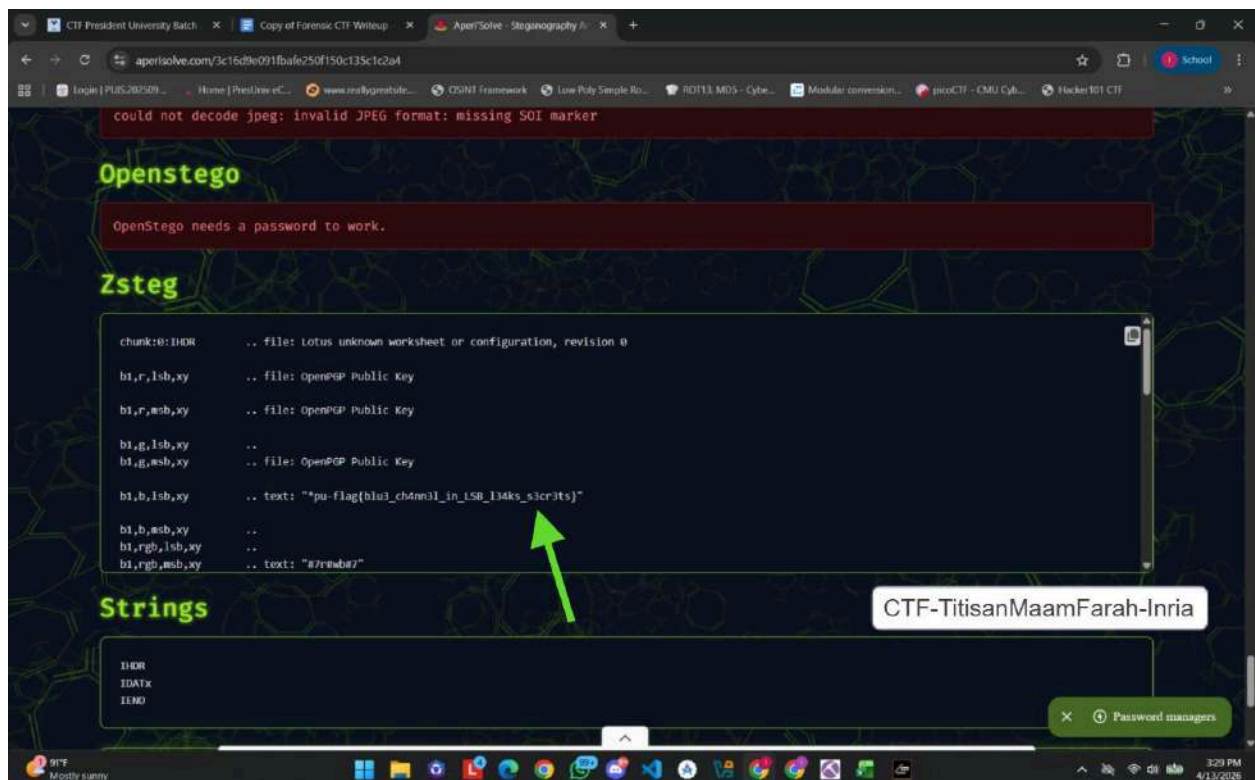
1. Aperi'Solve

Solving Step-by-step:

1. First we put the image to the Aperi'solve



2. Check one by one in Aperi'solve for the flag like exiftool, binwalk etc.
3. Until Zsteg we found the flag in there



TripleThreat2

Solved On: 13-4-2026

Solved by: Sheryl Sharon Kaparang

Flag Retrieved:

pu-flag{W3-1mpl3mEnt-tH15-1n-St3g0-n0w-t0-m4k3-y0u-B4LD-h3h3h3}

Challenges overview:

This challenge combines steganography and classical cryptography. The task was to extract hidden encrypted text from an image, identify the intended encryption methods from the provided clues, determine the correct key, and recover the final flag. The Python code was incomplete, so it served mainly as a reference for the cryptographic methods used.

Key Findings:

The challenge initially appeared to hide a message inside an image. Analysis showed that chall.png contained hidden data in its LSB bit planes, indicating steganography. The Python file also pointed to three classical ciphers:

Columnar Transposition Cipher, Atbash Cipher, and Caesar Cipher,

although the code itself was incomplete and only served as a clue. Another clue from the second image led to the correct key, "personanongrata".

Forensic Analysis:

1. **File inspection:** The challenge files were identified, especially chall.png, reyna2.jpg, and chall.py.
2. **Steganography analysis:** chall.png was analyzed using Aperi'solve, which revealed hidden encrypted text in the LSB section.
3. **Hidden text extraction:**
The encrypted text found was:
"iv0Ncbc3dWe3uc3-3--0-o-b-{Hs435vLdv-uS1-Dz3v3h-Kiq}11-hjb00z-4h"
4. **Code analysis:** The Python script was reviewed and showed three methods: Columnar Transposition Cipher, Atbash Cipher, and Caesar Cipher.
5. **Key identification:** Since the original key was redacted, another clue was analyzed until the correct key, personanongrata, was found.
6. **Decryption process:** The encrypted text was decrypted using the correct key and reverse order of the algorithms until the final flag was obtained.

Tools Used:

1. Aperi'solve (zsteg, pcrt, decomposer)
2. VSCode (python)
3. Duplicheker

Solving Step-by-step:

1. In the .zip folder there are 3 files, chall.py, chall.png, reyna2.jpg.

 chall	Python Source File
 chall	PNG File
 reyna2	JPG File

CTF-TITISANMAAMFARAH-SHERYL

2. I check every existing file one by one, starting from chall.py. I started by opening chall.py to review the provided script. The code was incomplete and some important values were still redacted, so I needed further analysis to understand its intended logic.

```
1  key = "REDACTED"
2  text = "REDACTED"
3  shift = len(key)
4
5  # Encrypt 1
6  key_order = sorted(range(len(key)), key=lambda k: key[k])
7  columns = ['' for _ in key]
8  for idx, char in enumerate(text):
9      columns[idx % len(key)] += char
10
11  ciphertext = ''
12  for idx in key_order:
13      ciphertext += columns[idx]
14
15  # Encrypt 2
16  c = ''
17  if 'A' <= c <= 'Z':
18      c = chr(ord('Z') - (ord(c) - ord('A')))
19  elif 'a' <= c <= 'z':
20      c = chr(ord('z') - (ord(c) - ord('a')))
21
22
23  # Encrypt 3
24  result = []
25  for char in text:
26      if char.isalpha():
27          base = ord('A') if char.isupper() else ord('a')
28          result.append(chr((ord(char) - base + shift) % 26 + base))
29      else:
30          result.append(char)
```

CTF-TITISANMAAMFARAH-SHERYL

3. Since the original chall.py was incomplete, I used ChatGPT only as a supporting tool to understand the coding logic, reconstruct the decryption script, and assist in testing the decryption process.

```
else:  
    result.append(char)
```

what does this code mean?

Selesai berpikir selama 6s

This code is showing the idea of a 3-layer encryption process, but it is incomplete.

Here is what each part means.

1. Key and shift

CTF-TITISANMAAMFARAH-SHERYL

Python



Jalankan

```
key = "REDACTED"  
text = "REDACTED"  
shift = len(key)
```

- `key` is the secret key

Decrypt code:

Python

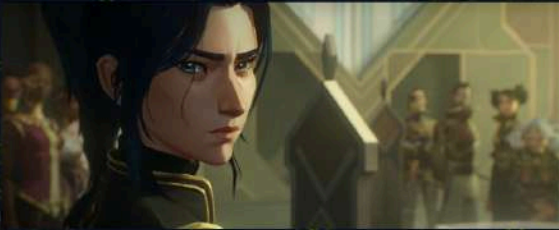


Jalankan

```
key = "REDACTED"  
ciphertext = "REDACTED"  
  
def columnar_decrypt(ciphertext, key):  
    n = len(key)  
    key_order = sorted(range(n), key=lambda k: key[k])  
  
    L = len(ciphertext)  
    rows = L // n  
    extra = L % n  
    col_lens = [rows + (1 if i < extra else 0) for i in range(n)]  
  
    cols = [''] * n  
    pos = 0  
    for idx in key_order:  
        clen = col_lens[idx]  
        cols[idx] = ciphertext[pos:pos + clen]  
        pos += clen  
  
    plaintext = []  
    max_len = max(len(c) for c in cols)  
    for r in range(max_len):
```

CTF-TITISANMAAMFARAH-SHERYL

4. Next, because the other 2 files are image files, so I immediately uploaded them to aperi solve for more in-depth analysis.
Starting from chall.png:

CTF-TITISANMAAMFARAH-SHERYL

First upload:	Mon, 13 Apr 2026 03:54:04 GMT
Last upload:	Mon, 13 Apr 2026 06:29:07 GMT
Name(s):	chall[1].png, prt_recovered_3bd6c974e4eb9236b6a4bd7499d9e1f7.png, chall.png, chall-fixed.png
Size:	1.71 MB
Upload count:	5
Common password(s):	PEOPLE WHO ARE WE WANTED WHEN A COUNTRY X

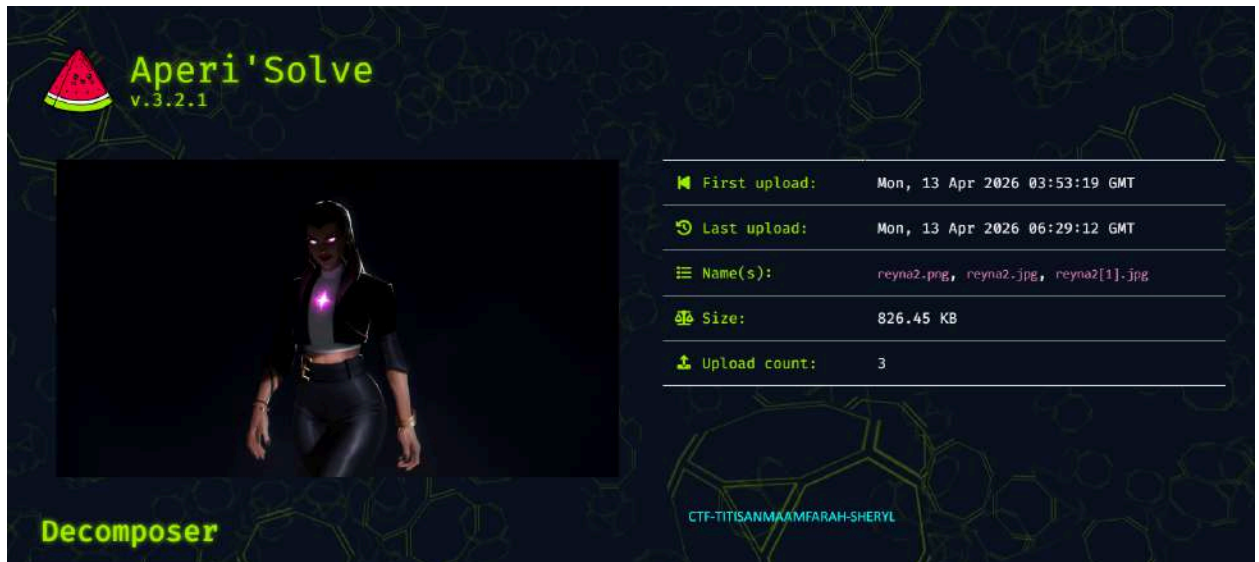
I scrolled down until I found something unusual in the zsteg section, I found a hidden encrypted text:
 “iv0Ncbc3dWe3uc3-3--0-o-b-{Hs435vLdv-uS1-Dz3v3h-Kiq}11-hjb00z-4h”(I answered this encrypted text in challenge part 2 and it is correct).

Zsteg

b1,bgr,msb,xy	..
b1,rgba,lsb,xy	.. text: <u>“iv0Ncbc3dWe3uc3-3--0-o-b-{Hs435vLdv-uS1-Dz3v3h-Kiq}11-hjb00z-4h”</u>
b1,rgba,msb,xy	.. file: OpenPGP Secret Key
b1,abgr,lsb,xy	..
b1,abgr,msb,xy	.. file: OpenPGP Secret Key
b2,r,lsb,xy	.. file: a.out VAX demand paged (first page unmapped) pure executable not stripped
b2,r,msb,xy	..
b2,g,lsb,xy	..
b2,g,msb,xy	..
b2,b,lsb,xy	.. text: “EAUUAETTDEEE”

CTF-TITISANMAAMFARAH-SHERYL

5. And then, continue uploading the reyna2.jpg file



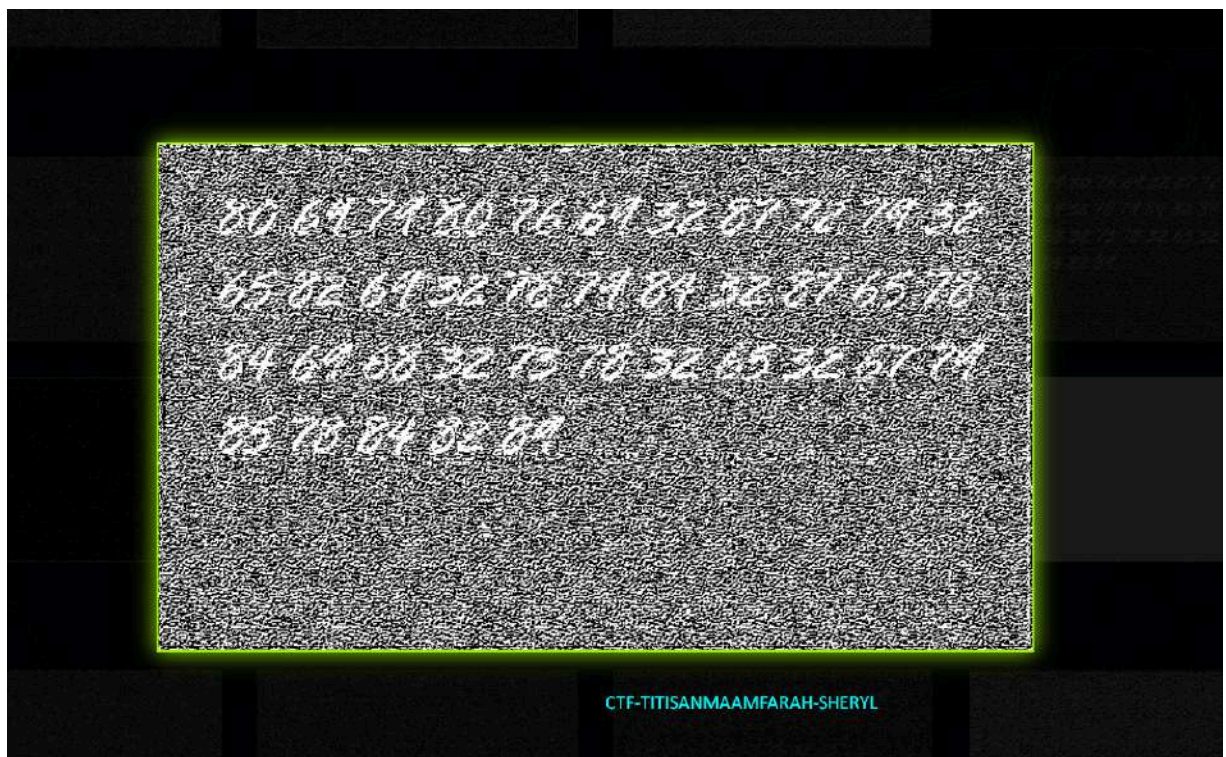
I also checked the PNG structure using the PcrT feature in Aperi'solve. The result showed that the PNG header was inconsistent and had to be repaired, after which the file was successfully fixed and saved. This confirmed that the image had been manipulated and was worth analyzing further for hidden content.



And then I tried uploading it again in aperisolve because I felt something was suspicious



After the PNG was repaired, I checked the extracted results in Aperi'solve. These outputs showed that the image had multiple hidden layers, so I continued the steganography analysis.



6. From the extracted result, I suspected that the hidden clue might be a text-based or ASCII-related clue, so I continued the analysis from that assumption.

ASCII ▼ To Text ▼

CTF-TITISANMAAMFARAH-SHERYL

80 69 79 80 76 69 32 87 72 79 32 65 82 69 32 78 79 84 32 87 65 78 84 69 68 32 73 78 32 65 32 67 79 85 78 84 82 89

PEOPLE WHO ARE NOT WANTED IN A COUNTRY

Sample Convert

After obtaining the numeric clue, I converted it from ASCII to text using duplichecker. The result was “PEOPLE WHO ARE NOT WANTED IN A COUNTRY,” which led me to the correct key.

7. From the conversion results, I tried searching on Google and tried the words one by one until I found one that managed to raise the flag.

Google PEOPLE WHO ARE NOT WANTED IN A COUNTRY called?

Semua Gambar Berita Video Shopping Video singkat Lainnya ▾ Alat ▾

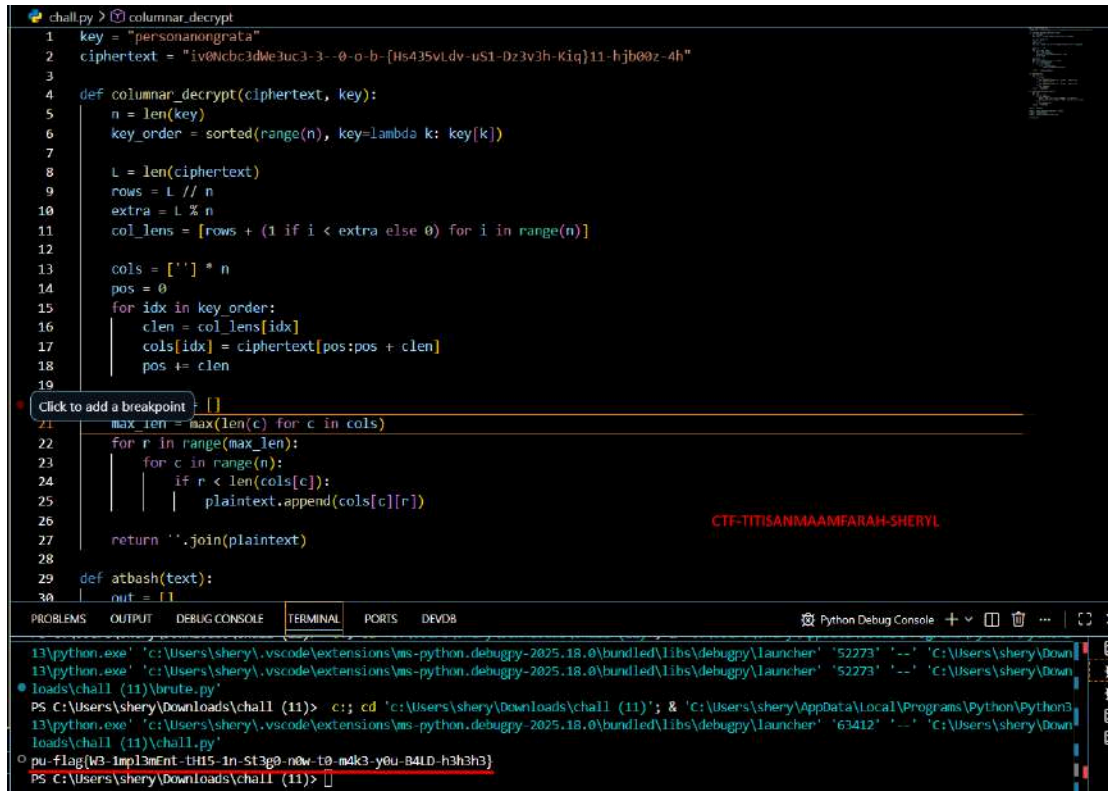
Ringkasan AI Indonesia CTF-TITISANMAAMFARAH-SHERYL

People who are not wanted, typically due to illegal status, criminal activity, or threats to national security, are commonly called **deportees** or **undesirable aliens**. If formally expelled, they are often referred to as persons in **exile** or under **banishment**, and if hiding from justice, they are known as **fugitives**. [USA.gov](#) +4

Common Terms and Contexts:

- **Deportee / Prohibited Immigrant:** A person who has no lawful right to stay in a country and is being removed.
- **Persona non grata:** A diplomatic term for a foreign person whose entering or remaining in a particular country is forbidden by that country's government.
- **Exile / Banishment:** Refers to being expelled from one's native country, often due to political reasons or being considered undesirable by the state.

And finally the key to successfully displaying the flag is personanongrata.



```
chall.py > columnar_decrypt
1 key = "personanongrata"
2 ciphertext = "iv0WcbC3dMe3uc3-3--0-o-b-[Hs435vLdv-uS1-0z3v3h-Kiq]11-hjb00z-4h"
3
4 def columnar_decrypt(ciphertext, key):
5     n = len(key)
6     key_order = sorted(range(n), key=lambda k: key[k])
7
8     L = len(ciphertext)
9     rows = L // n
10    extra = L % n
11    col_lens = [rows + (1 if i < extra else 0) for i in range(n)]
12
13    cols = [''] * n
14    pos = 0
15    for idx in key_order:
16        clen = col_lens[idx]
17        cols[idx] = ciphertext[pos:pos + clen]
18        pos += clen
19
20    Click to add a breakpoint
21    max_len = max(len(c) for c in cols)
22    for r in range(max_len):
23        for c in range(n):
24            if r < len(cols[c]):
25                plaintext.append(cols[c][r])
26
27    return ''.join(plaintext)
28
29 def atbash(text):
30    out = []
31
32    13\python.exe 'c:\Users\shery\.vscode\extensions\ms-python.debugpy-2025.18.0\bundle\libs\debugpy\launcher' '52273' '--' 'C:\Users\shery\Down
33\python.exe' 'c:\Users\shery\.vscode\extensions\ms-python.debugpy-2025.18.0\bundle\libs\debugpy\launcher' '52273' '--' 'C:\Users\shery\Down
34loads\chall (11)\brute.py'
35PS C:\Users\shery\Downloads\chall (11)> c:; cd 'C:\Users\shery\Downloads\chall (11)'; & 'C:\Users\shery\AppData\Local\Programs\Python\Python3
3613\python.exe' 'c:\Users\shery\.vscode\extensions\ms-python.debugpy-2025.18.0\bundle\libs\debugpy\launcher' '63412' '--' 'C:\Users\shery\Down
37loads\chall (11)\chall.py'
38o pu-flag{W3-1mpl3mEnt-tiH5-1n-St3g0-n0w-t0-m4k3-y0u-B4LD-h3h3h3}
39PS C:\Users\shery\Downloads\chall (11)>
```

that's the flag at the bottom.

Nightmare

Solved On: 13/04/2026

Solved by: Richie Frederico Wong

Flag Retrieved: pu-flag{2025-01-07_01:10:13_lsass.exe_WLDAP32.dll}

Challenges overview:

This is a log analysis challenge. We're given a Windows Application Event Log (XML format) and asked to find traces of a specific LDAP vulnerability. Think of it like being a detective > We're sifting through system logs to find evidence of an attack.

Key Findings:

What immediately stood out was that among the hundreds of routine log entries, there was one anomalous crash event that involved `lsass.exe` — a critical Windows security process that handles authentication. What made it suspicious was that the crash was caused by `WLDAP32.dll`, which is the library responsible for LDAP client operations. A crash in this specific combination is a well-known indicator of the **LDAPNightmare exploit (CVE-2024-49113)**, a denial-of-service vulnerability that was actively being discussed in early January 2025, right around the timestamp of the event.

Forensic Analysis:

I opened the XML event log file in Notepad++ and searched for the keyword **LDAP** — since Part 1 of the challenge already gave context about LDAP being involved. That immediately surfaced `WLDAP32.dll`, and right above it was the faulting application `lsass.exe`. On the same line, the `TimeCreated` field gave the exact timestamp of the crash: **2025-01-07 at 01:10:13 UTC**.

Cross-referencing with public CVE databases confirmed this matches the signature of **CVE-2024-49113**, where an attacker sends a crafted CLDAP referral packet causing LSASS to crash and the system to reboot.

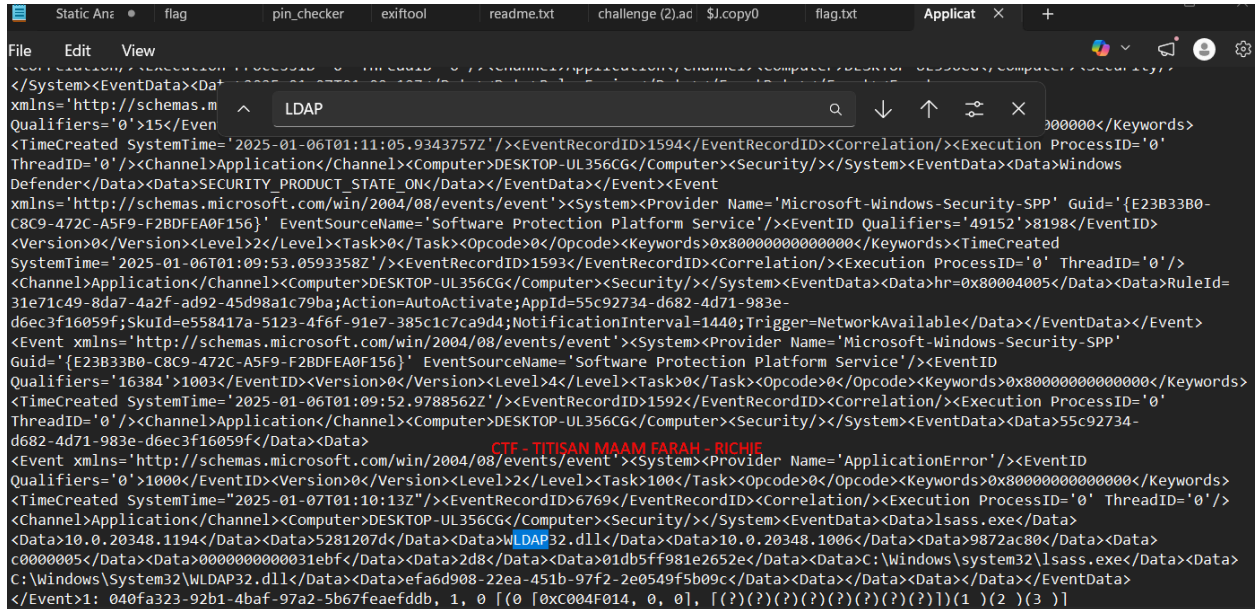
Tools Used:

Notepad++ > to open and read through the raw XML event log

Solving Step-by-step:

1. I directly open the notepad and find the keyword LDAP bcs from the part 1 it always mentioned the LDAP and after that in notepad we can

To find the timecreated, just see it in the same line because it's clearly said TimeCreated = ...



pu-flag{YYYY-MM-DD_HH:MM:SS_FaultyAppName_FaultyModuleName
} e.g pu-flag{2000-01-01_12_00_00_test.exe_test.dll} and then I construct
all the information I got to become

Donee, we got the correct answer.

lost-d4-Imp0rt4nt-Fil3

Solved On: 13-4-2026

Solved by: Inria Altje Kalalo

Flag Retrieved: pu-flag{2025-01-27 23:26:42_126638}

Challenges overview:

In this challenge we have to find the last updated timestamp of the file as well as MFT entry number and then the file containing the flag was securely deleted by a malicious actor.

Key Findings:

What is interesting in this challenge is when we successfully parsed the provided UsnJrnl (\$J) file and identified a total of over 300,000 file system entries, indicating significant system activity. Initial observations revealed multiple file operations such as file creation, deletion, and renaming events. Notably, the presence of faulty application logs suggests that an application crashed during execution, which is a key indicator of abnormal or potentially suspicious behavior. This anomaly is considered important as it may be directly related to the event timeline required to construct the flag.

Forensic Analysis:

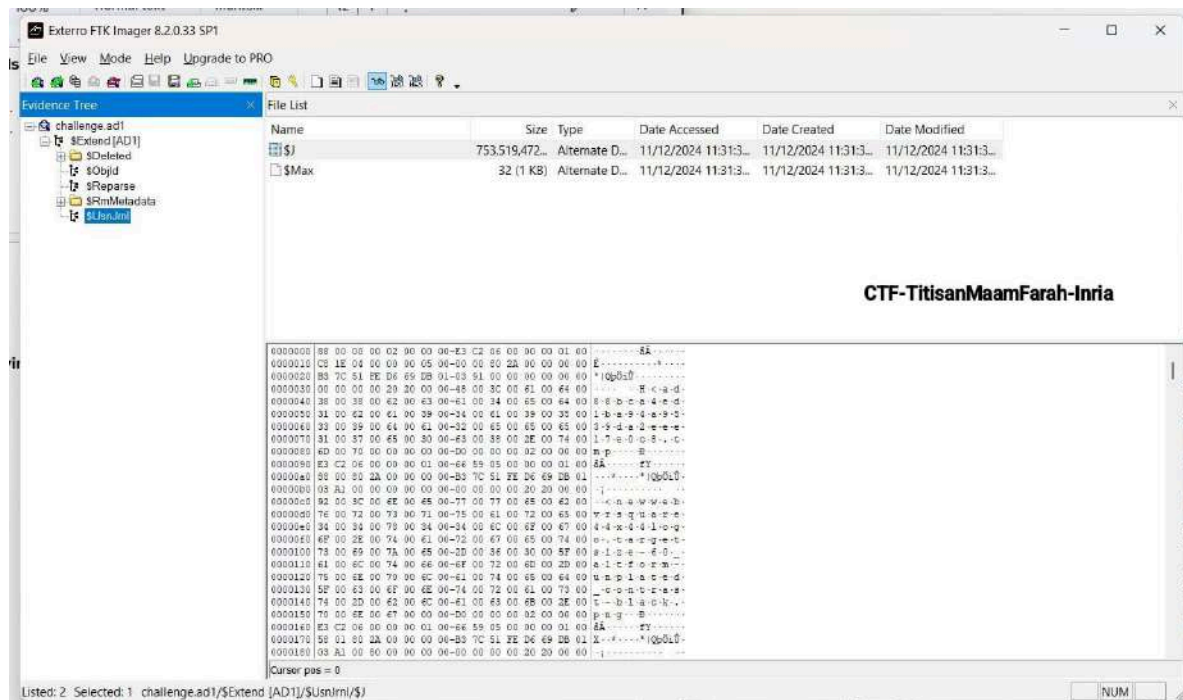
1. Parsed the UsnJrnl (\$J) file using MFTECmd.
2. Generated and reviewed the CSV output.
3. Analyzed file system activities such as create, delete, and rename events.
4. Focused on entries related to application crashes (faulty application).
5. Identified the timestamp of the event.
6. Extracted the faulty application name and faulty module name.
7. Used the collected information to construct the flag.

Tools Used:

1. FTK Imager
2. MFTECmd
3. Microsoft Excel

Solving Step-by-step:

1. The first step is we use FTK Imager for open the file that we get from the challenge. How do i know that is use FTK Imager? Its because the file name is .ad1
2. When we open the file disk in FTK Imager we can see in the bottom of the root there is \$RmMetadata inside is \$UsnJml and inside that we can see the file name \$J, That's the file that we need to extract.



3. After we extract the file, we need to convert the file into a bin file and then because of the file is still a raw log we need to use more tool for parser it. we can use EricZimmerman tool exactly the MFTECmd to convert it into a readable csv. Use this command
MFTECmd.exe -f UsnJrnL_J.bin --csv output/

```
Command Prompt
--fls      When true, displays contents of directory from $MFT specified by --de. Ignored when --de points to a
           file [default: False]
--ds <ds>  Dump full details for Security Id from $SDS as decimal or hex. Example: 624 or 0x270
--dt <dt>  The custom date/time format to use when displaying time stamps. See https://goo.gl/CNVq0k for
           options [default: yyyy-MM-dd HH:mm:ss.ffffff]
--sn       Include DOS file name types in $MFT output [default: False]
--fl       Generate condensed file listing of parsed $MFT contents. Requires --csv [default: False]
--at       When true, include all timestamps from 0x30 attribute vs only when they differ from 0x10 in the $MFT
           [default: False]
--rs       When true, recover slack space from FILE records when processing $MFT files. This option has no
           effect for $I30 files [default: False]
--vss      Process all Volume Shadow Copies that exist on drive specified by -f [default: False]
--dedupe   Deduplicate -f & VSCs based on SHA-1. First file found wins [default: False]
--debug    Show debug information during processing [default: False]
--trace    Show trace information during processing [default: False]
--ir       Include resident data in JSON/CSV output [default: False]
--re <re>  Comma-separated list of extensions to include for resident data (e.g., '.txt,.ps1,.bat'). If
           omitted, includes all
--rm <rm>  Maximum size in bytes for resident data to include (default: 1024, max: 1024000) [default: 1024]
--version  Show version information
-?, -h, --help  Show help and usage information

C:\Users\Lenovo\Downloads\MFTECmd>MFTECmd.exe -f UsnJrnl_J.bin --csv output/
MFTECmd version 1.3.0.0

Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/MFTECmd

Command line: -f UsnJrnl_J.bin --csv output/
Warning: Administrator privileges not found!
File type: UsnJournal

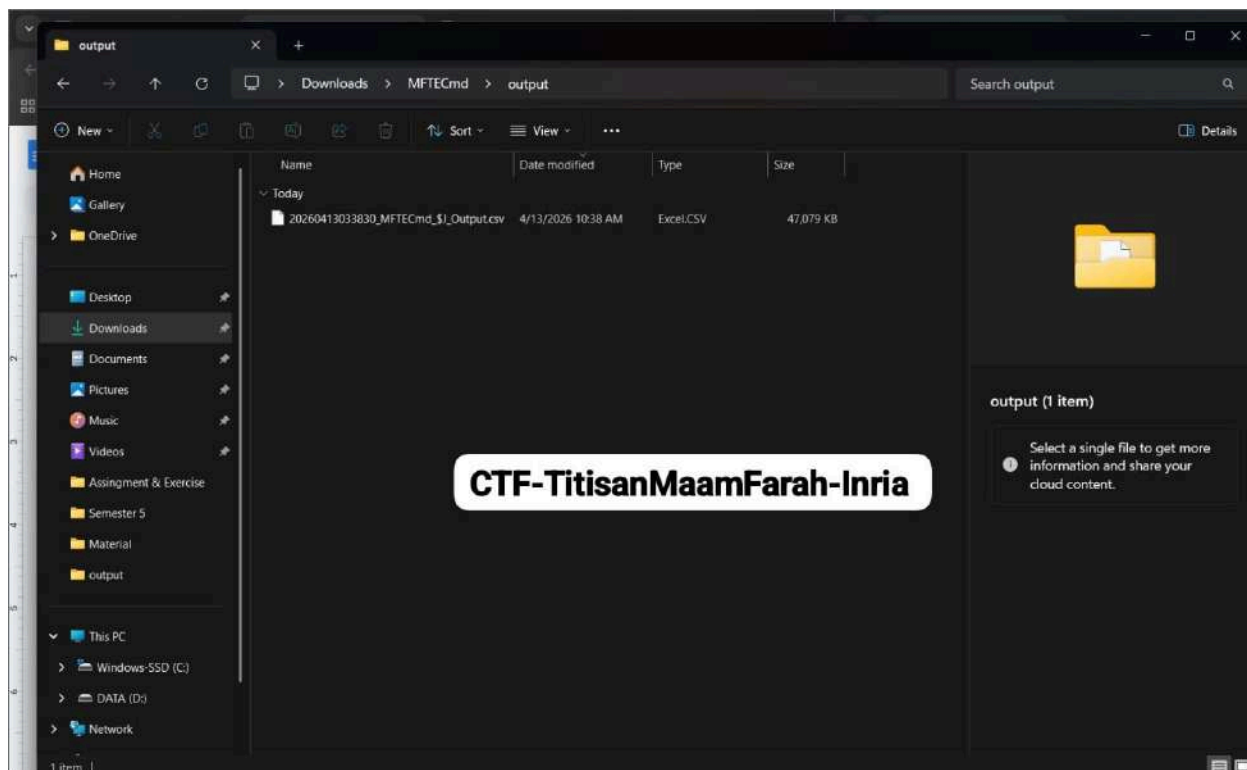
Processed UsnJrnl_J.bin in 1.8363 seconds

Usn entries found in UsnJrnl_J.bin: 309,552
Path to output/ doesn't exist. Creating...
CSV output will be saved to output\20260413033830_MFTECmd_$J_Output.csv

C:\Users\Lenovo\Downloads\MFTECmd>
```

CTF-TitisanMaamFarah-Inria

4. After that the csv file will automatically saved in MFTECmd output folder



CTF-TitisanMaamFarah-Inria

- Microsoft Word 2016 - [20260413033830_MFTECmd.\$1 Output]

File Home Insert Draw Page Layout Formulas Data Review View Help Acrobat

Font: Aptos Narrow, 11, A, A, B, I, U, Color, Background Color, Alignment, Spacing, Language, Merge & Center, General, Conditional Formatting, Table, Cell, Styles, Insert, Delete, Format, AutoSum, Fill, Sort & Filter, Find & Select, Add-ins, Comments, Share

Find and Replace: Find what: Flag, Replace with: Flag, Options >>, Find All, Find Next, Close

	A	B	C	D	E	F	G	H	I	J	K
202217	edb8627a-9de6-496a-a429-139da185.tmp	127015	14	104257	1			752449744	23:16.6 FileCreate		Archive
202218	edb8627a-9de6-496a-a429-139da185.tmp	127015	14	104257	1			752449688	23:16.6 DataExtend FileCreate		Archive
202219	edb8627a-9de6-496a-a429-139da185.tmp	127015	14	104257	1						Archive
202220	edb8627a-9de6-496a-a429-139da185.tmp	127015	14	104257	1						Archive
202221	edb8627a-9de6-496a-a429-139da185.tmp	127015	14	104257	1						Archive
202222	Preferences-RF5307.d.TMP	134548	12	104257	1						Hidden Archive Temporary
202223	Preferences-RF5307.d.TMP	134548	12	104257	1						Hidden Archive Temporary
202224	Preferences-RF5307.d.TMP	134548	12	104257	1						Hidden Archive Temporary
202225	Preferences	134431	11	104257	1						Archive
202226	Preferences-RF5307.d.TMP	134431	11	104257	1						Archive
202227	edb8627a-9de6-496a-a429-139da185.tmp	127015	14	104257	1						Archive
202228	Preferences	127015	14	104257	1						Archive
202229	Preferences-RF5307.d.TMP	134431	11	104257	1						Archive
202230	Preferences	127015	14	104257	1			752451344	23:16.6 SecurityChange RenameNewName BasicInfoChange		Archive
202231	Preferences-RF5307.d.TMP	134431	11	104257	1			752451432	23:16.6 FileDelete Close		Archive
202232	New Text Document.txt	126638	7	103558	2			752451564	23:22.3 RenameOldName		Archive
202233	flag.txt	126638	7	103558	2			752451688	23:22.3 RenameNewName		Archive
202234	flag.txt	126638	7	103558	2			752451768	23:22.3 RenameNewName Close		Archive
202235	01b4d95c15532a.automatic.Destination	104250	3	104911	7			752451848	23:22.3 DataOverwrite		Archive
202236	01b4d95c15532a.automatic.Destination	104250	3	104911	7			752451992	23:22.3 DataOverwrite Close		Archive
202237	flag.txt	126638	7	103558	2			752452136	23:22.3 ObjectIDChange		Archive
202238	flag.txt	126638	7	103558	2			752452216	23:22.3 ObjectIDChange Close		Archive
202239	flag.txt.link	134431	12	103627	1			752452296	23:22.3 FileCreate		Archive
202240	flag.txt.link	134431	12	103627	1			752452384	23:22.3 DataExtend FileCreate		Archive
202241	flag.txt.link	134431	12	103627	1			752452472	23:22.3 DataExtend FileCreate Close		Archive
202242	57b7091e01b63767.automatic.Destination	104534	2	104911	7			752452560	23:22.3 DataOverwrite		Archive
202243	57b7091e01b63767.automatic.Destination	104534	2	104911	7			752452704	23:22.3 DataOverwrite DataExtend		Archive
202244	57b7091e01b63767.automatic.Destination	104534	2	104911	7			752452848	23:22.3 DataOverwrite DataExtend Close		Archive
202245	amp.d	105785	1	1416	1			752452992	23:23.0 DataOverwrite		Archive
202246	edb.chk	105783	1	1416	1			752453072	23:23.0 DataOverwrite		Archive
202247	edb.chk	105783	1	1416	1			752453152	23:23.0 DataOverwrite Close		Archive
202248	edb.chk	1057									

6. In the challenge to get the flag Search by the question. That is several file that have a same name, but we should check the activity and the timestamp, until i got the right flag. Before that i try several combination it turns out it is wrong until this file. We do asked maam for more hint, and maam said about the format is wrong.

File Name	Size	Date
302726 fe0d20b1-d83c-4e0d-872a-505e4b561.tmp	203721	4 109006
302729 Network Persistent State-RF82b9d.TM.TMP	203725	3 109006
302730 Network Persistent State-RF82b9d.TM.TMP	203725	3 109006
302731 Network Persistent State-RF82b9d.TM.TMP	203725	3 109006
302732 Network Persistent State-RF82b9d.TM.TMP	203521	3 109006
302733 Network Persistent State-RF82b9d.TM.TMP	203521	3 109006
302734 fe0d20b1-d83c-4e0d-872a-505e4b561.tmp	203721	4 109006
302735 Network Persistent State-RF82b9d.TM.TMP	203721	4 109006
302736 Network Persistent State-RF82b9d.TM.TMP	203521	3 109006
302737 Network Persistent State-RF82b9d.TM.TMP	203721	4 109006
302738 Network Persistent State-RF82b9d.TM.TMP	203521	3 109006
302739 ConsoleHost_history.txt	109498	5 109437
302740 ConsoleHost_history.txt	109498	5 109437
302741 flag.txt	126638	7 103558
302742 AAAA.AAA	126638	7 103558
302743 AAAA.AAA	126638	7 103558
302744 AAAA.AAA	126638	7 103558
302745 B88B.B88	126638	7 103558
302746 B88B.B88	126638	7 103558
302747 B88B.B88	126638	7 103558
302748 CCCC.CCC	126638	7 103558
302749 CCCC.CCC	126638	7 103558
302750 CCCC.CCC	126638	7 103558
302751 DDDD.DDD	126638	7 103558
302752 DDDD.DDD	126638	7 103558
302753 DDDD.DDD	126638	7 103558
302754 EEEE.EEE	126638	7 103558
302755 EEEE.EEE	126638	7 103558
302756 EEEE.EEE	126638	7 103558
302757 FFFF.FFF	126638	7 103558
302758 FFFF.FFF	126638	7 103558
302759 FFFF.FFF	126638	7 103558

Fred 13:03
nope we still have lost da important...
flag{2025-01-27_23:26:42_SDELETE64.EXE_flag.txt}

is this the right one

FaY 13:04
No

Fred 13:07
where can we find the faulty app maam
i mean we alr extract the csv but after we found flag.txt ...

FaY 13:16
Change that delete. Exe
With number

Fred 13:23
flag{2025-01-27_23:26:42_126638_flag.txt}

this?? it's incorrect ma'am

CTF-TitisanMaamFarah-Inria

- In here actually we kinda lost because of the example format flag that actually different with the actual flag. So after several trying and changes the format into this flag pu-flag{2025-01-27 23:26:42_126638} it turns out correct. Of course after a hint from maam that Richie asked.

goTTa-fix-the-corruptIOn

Solved On: 13/04/2026

Solved by: Richie Frederico Wong

Flag Retrieved: pu-flag{3asy-Obfusc4t1On-4nd-st3g-does-it?}

Challenges overview:

A PNG file of Vi from Arcane was provided in a corrupted/unreadable state. The challenge involved restoring the image to its original form and then discovering a hidden secret embedded within it using steganography.

Key Findings:

Upon first inspection, the file had a .png extension but could not be opened by any image viewer. Running a hex dump immediately revealed something unusual bcs every single byte in the file was one of only four values: 0x00, 0x01, 0x10, or 0x11. This is a dead giveaway that each byte encodes exactly 2 bits using a pseudo-binary decimal scheme, rather than storing normal byte values.

Forensic Analysis:

1. **File type check:** file vi.png returned data — not a valid image format.
2. **Hex analysis:** od -t x1z revealed all bytes were limited to {0x00, 0x01, 0x10, 0x11}.
3. **Encoding identification:** Each byte stores 2 bits — the upper nibble (0 or 1) and lower nibble (0 or 1) — expanding the original file 8×.
4. **Reconstruction:** Extracted both nibbles from each byte as bits, then packed every 8 bits back into a byte. The first 8 bytes of the result matched the PNG magic signature \x89PNG\r\n\x1a\n exactly.
5. **Steganography detection:** After restoring the PNG, LSB analysis of the pixel data revealed an **OpenStego signature** (0x1860) at offset 0 of the extracted bit stream, confirming a hidden file was embedded using OpenStego LSB steganography.
6. **Hidden content:** A JPEG file was found embedded starting at byte offset 745 of the LSB-extracted data, containing the confidential message / flag.

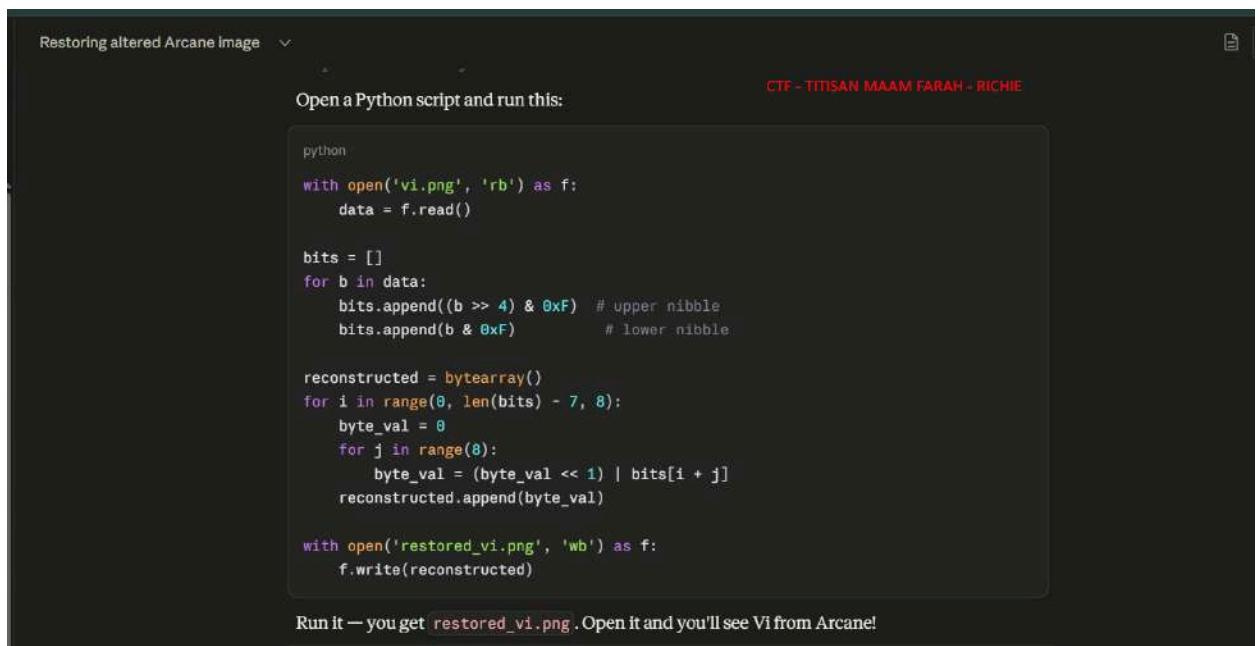
Tools Used:

- HxD app

- AperiSolve
- Python

Solving Step-by-step:

1. I Download and open vi.png in HxD. Immediately I notice something weird bcs every byte is only one of four values: 00, 01, 10, or 11. A normal PNG would start with 89 50 4E 47. This one doesn't, which means the file has been obfuscated at the byte level. After that, those four values (00, 01, 10, 11) are exactly the four possible 2-bit combinations. Each byte is storing only 2 bits of real data (in its upper and lower nibble). So the file is 8× bigger than it should be.
2. After I found out what is the problem I ask claude to make me a solver code for the image



The screenshot shows a chat interface with a dark theme. At the top, there's a title bar that says "Restoring altered Arcane Image" with a dropdown arrow. On the right, there's a small icon of a document. Below the title bar, there's a red text label that says "CTF - TITISAN MAAM FARAH - RICHIE". The main content area shows a prompt "Open a Python script and run this:" followed by a Python script. The script reads a file named 'vi.png', processes its data by extracting upper and lower nibbles, reconstructs the image, and saves it as 'restored_vi.png'. Below the script, there's a response from the AI: "Run it — you get restored_vi.png. Open it and you'll see Vi from Arcane!".

```
python
with open('vi.png', 'rb') as f:
    data = f.read()

bits = []
for b in data:
    bits.append((b >> 4) & 0xF) # upper nibble
    bits.append(b & 0xF)        # lower nibble

reconstructed = bytearray()
for i in range(0, len(bits) - 7, 8):
    byte_val = 0
    for j in range(8):
        byte_val = (byte_val << 1) | bits[i + j]
    reconstructed.append(byte_val)

with open('restored_vi.png', 'wb') as f:
    f.write(reconstructed)
```

Run it — you get `restored_vi.png`. Open it and you'll see Vi from Arcane!

3. After that, I got my restored_vi.png in my download folder, once i got it, I input it to aperi solve

Decompiler Superimposed

CTF - TITISAN MAAM FARAH - RICHIE



And thennn I got the flag from the superimposed by aperi solveee, donee.

latte

Solved On: 13-4-2026

Solved by: Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{h0w-d0-y0u-f1nd-m3}

Challenges overview:

The main goal in this challenge is to inspect the given image file, identify whether there is hidden data inside it, and recover the hidden flag. The challenge required careful observation of the image structure and the decomposed image outputs, because the flag was not visible in the normal image view.

Key Findings:

At first glance, the image looked like a normal coffee image. However, after checking it with forensic tools, I noticed several suspicious indicators. The file had multiple uploads, unusual PNG structure information, and decomposed image results that suggested hidden layered content. The most interesting part was that the hidden text did not appear directly in the original image, but became visible after color decomposition and superimposed analysis.

Forensic Analysis:

1. **File inspection:** Checked the image with Aperi'Solve and found suspicious decomposition results.
2. **PNG analysis:** Used PCRT and found a wrong PNG header and extra bytes after IEND.
3. **Flag extraction:** Inspected the superimposed layers and recovered the hidden flag.

Tools Used:

1. Aperi'solve (pcrt, decomposer)

Solving Step-by-step:

1. I checked the image with Aperi'Solve to inspect its metadata and decomposed layers for any hidden clues.


Aperi'Solve
v.3.2.1



First upload: Mon, 13 Apr 2026 03:53:44 GMT

Last upload: Mon, 13 Apr 2026 09:13:07 GMT

Name(s): coffee-latte.png, coffee-latte (1).png

Size: 800.28 KB

Upload count: 3

CTF-TITISANMAAMFARAH-SHERYL

Decomposer
Superimposed



- I continued scrolling and found the PCRT output, which showed a broken PNG header and extra bytes after the IEND chunk, suggesting hidden or altered data inside the file.

Pcrt

CTF-TITISANMAAMFARAH-SHERYL

Wrong PNG header: FFD8FFD800430004

Fixed header to: 89504f47000a1a0a

Correct IHDR CRC at offset 0x1285a

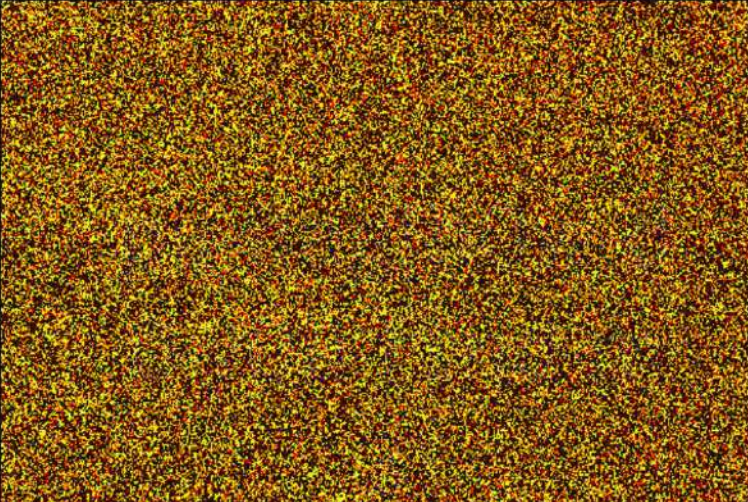
IHDR chunk check complete at offset 0x12845

IDAT chunk check complete at offset 0x1285e

Correct IEND chunk

Found 22 bytes after IEND: b'PK\x05\x06\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'

IEND chunk check complete



- After that, I saved the second extracted image and uploaded it again to continue the analysis, since it might contain another hidden layer or clue.

And yup, you can immediately see the flag in the decomposer section.

The screenshot displays the Aperi'Solve v.3.2.1 web application interface. At the top left is a watermelon logo. The main area features a large, square, noisy image with a yellow and brown pixelated pattern. To the right of this image is a metadata panel with the following details:

- First upload:** Mon, 13 Apr 2026 04:47:50 GMT
- Last upload:** Mon, 13 Apr 2026 09:37:13 GMT
- Name(s):** pcrf_recovered_dcf11b61b6842a2b0d7b5b9135e47.png, 00000140.png, inner.png
- Size:** 725.45 KB
- Upload count:** 4

Below the main image, the text "Decomposer Superimposed" is visible. A red arrow points to a row of four small, square, noisy images. Each of these images contains the text "pu-flag(h0w-d 0-y0u-f1nd-m 3)" in a different color (purple, red, magenta, and green).

CTF-TITISANMAAMFARAH-SHERYL

EzTraverse

Solved On: 13/04/2026

Solved by: Jesynta Ivolairya Harya

Flag Retrieved: pu-flag{p3rs1st3nc3_v14_cr0nd}

Challenges overview:

This challenge simulates a real-world forensic investigation where we were given a raw Linux disk image of a compromised web server and tasked with analyzing its filesystem using Autopsy to uncover how the attacker broke in through a disguised PHP web shell, escalated privileges to root, planted multiple persistence mechanisms, and ultimately find the hidden flag inside a malicious cron job file.

Key Findings:

- A file named sysupdate was hiding inside /etc/cron.d/, deliberately named to blend in as a legitimate system process, but contained a bash reverse shell connecting back to 10.10.14.23 every minute.
- A PHP file named img_jpg.php was found inside /var/www/html/uploads/, disguised as an image using a GIF magic bytes trick, but functioned as a fully working web shell capable of executing any system command.
- A hidden directory /tmp/.9b3f/ contained a persistence loader script designed to silently reinstall the cron backdoor if it was ever removed — showing the attacker planned ahead for detection.
- A rogue user account d43m0n666 was found in /etc/passwd with UID=0, granting it full root-level access without raising obvious suspicion.
- The attacker's SSH public key was injected into /root/.ssh/authorized_keys, allowing passwordless remote access at any time.
- /var/log/auth.log had been completely wiped so this is a clear sign the attacker attempted to cover their tracks after the compromise.

Forensic Analysis:

- Filesystem Analysis — Loaded challenge.img into Autopsy for visual navigation, and used debugfs to extract file contents directly from the image without mounting.
- Directory Enumeration — Listed contents of suspicious directories including /etc/cron.d/, /root/.ssh/, /tmp/.9b3f/, and /var/www/html/uploads/ to spot anomalous files.
- Cron Job Analysis — Read /etc/cron.d/sysupdate and found a malicious cron entry running a reverse shell as root every minute.
- Web Shell Analysis — Examined /var/www/html/uploads/img_jpg.php, a PHP web shell disguised as an image using GIF magic bytes, confirmed as the initial access vector.
- User Account Analysis — Reviewed /etc/passwd and /etc/shadow, identifying rogue account d43m0n666 with UID=0 granting full root privileges.
- SSH Key Analysis — Inspected /root/.ssh/authorized_keys and found the attacker's injected public key enabling passwordless SSH access.
- Hidden Script Analysis — Read /tmp/.9b3f/run, a hidden persistence loader that auto-reinstalls the cron backdoor if removed.
- Log Analysis — Reviewed auth.log, syslog, apache2/access.log, and bash history files to reconstruct the full attack timeline and confirm log wiping by the attacker.

Tools Used:

- Autopsy
- WSL

Solving Step-by-step:

Step 1 — Load the Disk Image into Autopsy

Open Autopsy and create a new case, then add challenge.img as a data source. Once ingested, Autopsy will parse the filesystem and display the full directory tree on the left panel.

Step 2 — Browse the Filesystem Tree

Navigate through the directory tree in Autopsy and explore all key directories. The following suspicious locations were identified:

/etc/cron.d/

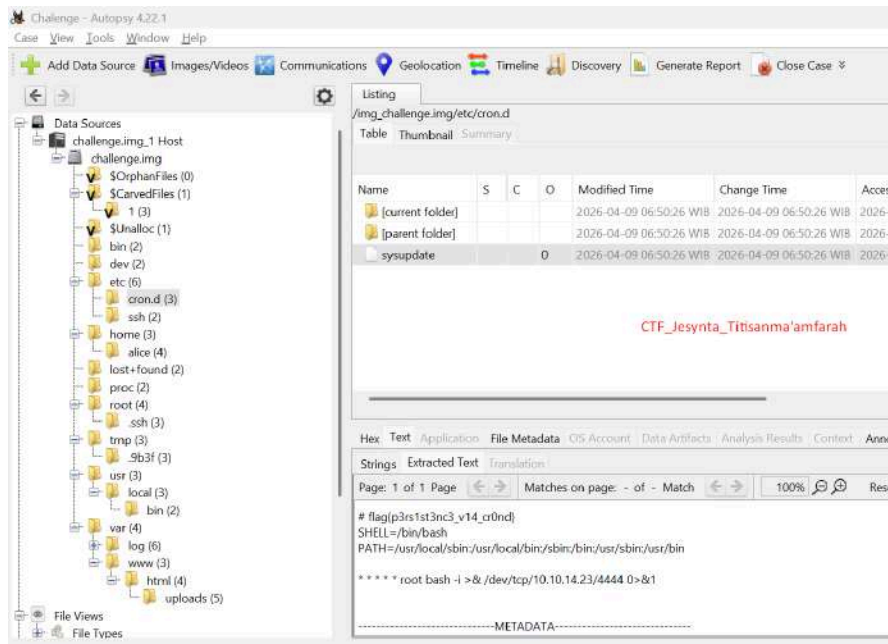
/root/.ssh/

/tmp/.9b3f/

/var/www/html/uploads/

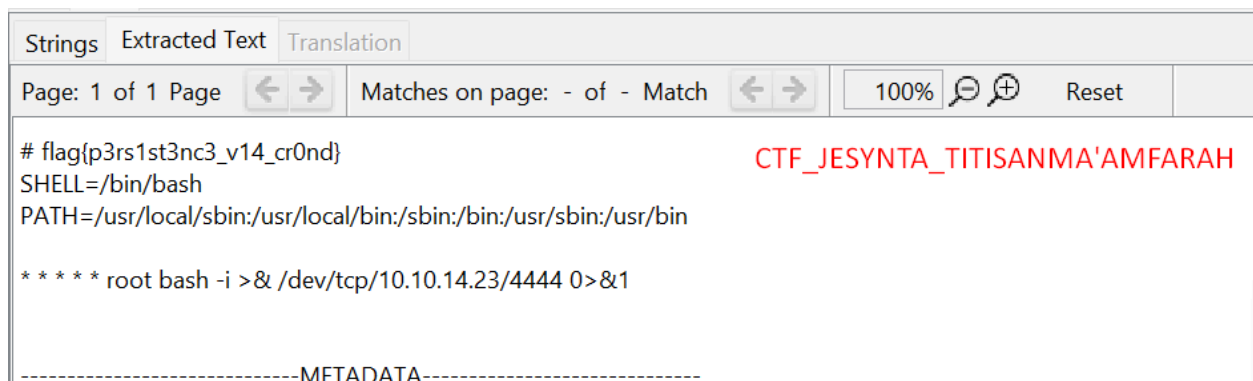
/var/log/

/home/alice/



Step 3 — Investigate /etc/cron.d/

Click on /etc/cron.d/ in the Autopsy tree. A file named sysupdate appears — suspicious because it mimics a legitimate system update task. Click the file and view its content in the bottom text panel.



The flag is embedded as a comment at the very top of the file.

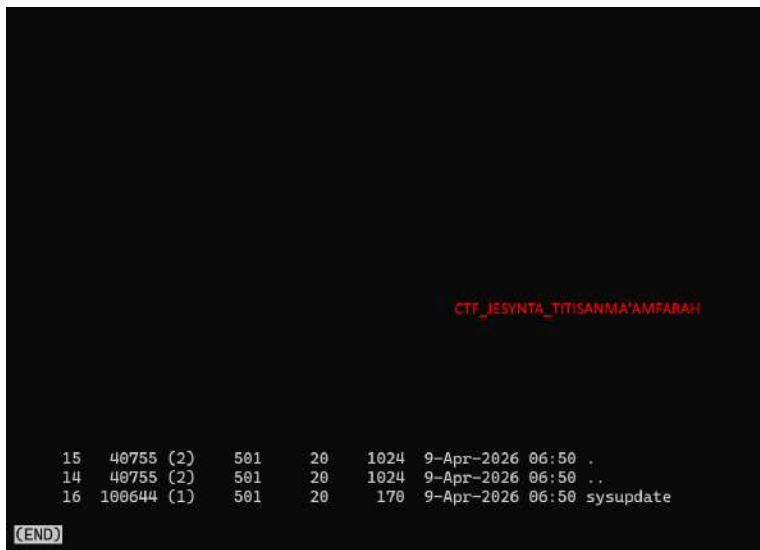
Step 4 — Confirm via debugfs (CLI Verification)

To verify the finding outside of Autopsy, use debugfs to directly read the file from the disk image:

```
debugfs -R "ls -l /etc/cron.d" challenge.img
```

Output:

```
16 100644 501 20 170 8-Apr-2026 sysupdate
```



Step 5 — Submit the Flag

Copy the flag found inside /etc/cron.d/sysupdate and submit it on the challenge website:

```
Pu-flag{p3rs1st3nc3_v14_cr0nd}
```

MailMailSnailMail

Solved On: 13/04/2026

Solved by: Richie Frederico Wong

Flag Retrieved: pu-flag{Th15-1s-4n-4tt3mpt3d-Ph1sh1ng-Y0u-H4v3-LE4rned}

Challenges overview:

This challenge simulates a real-world phishing investigation. We were given a suspicious .eml file, an SMTP log, and had to trace the email's true origin, identify forgeries in the mail routing chain, and extract a hidden flag from a fake PDF attachment.

Key Findings:

The first thing that caught my eye was the sender domain internalc0rp.net with a zero instead of the letter O. It looked almost identical to the real domain at first glance. The Reply-To also pointed somewhere completely different from the display name, which was a big red flag.

Forensic Analysis:

- Analyzed email headers manually to trace the Received chain and identify the true originating IP
- Checked SPF, DKIM, and DMARC results in the Authentication-Results header
- Decoded the Base64 token parameter in the phishing URL to reveal victim-specific data
- Decoded the Message-ID header which contained a hidden clue

Tools Used:

- Notepad > reading the raw .eml
- CyberChef > quick decode verification

Solving Step-by-step:

1. Opened the .eml in a text editor and read through the headers

5. Noticed the Message-ID had a suspicious encoded segment especially it says CLUE so I, decoded it > revealed check_the_attachment

```
Static A • flag pin_checker exiftool readme.txt challenge (2): $J.copy0 flag.txt Application.xt challenger X +
File Edit View
for <j.harris@internalcorp.net>; Mon, 07 Apr 2025 09:14:26 -0500 (CDT)
MIME-Version: 1.0
Date: Mon, 07 Apr 2025 14:14:25 +0000
Message-ID: <CLUE3-Y2h1Y2tfDgh1X2F0dGFjaG11bnQ=@securepayments-verify.com>
From: "IT Security - InternalCorp" <security-noreply@internalcorp.net>
Reply-To: harvest@securepayments-verify.com
To: j.harris@internalcorp.net
Subject: [ACTION REQUIRED] Verify Your Account - Unusual Login Detected
X-Mailer: Microsoft Outlook 16.0
X-Originating-IP: 185.220.101.47
Authentication-Results: mx.internalcorp.net;
    spf=fail (sender IP is 185.220.101.47; expected internalcorp.net) smtp.mailfrom=internalcorp.net;
    dkim=none;
    dmarc=fail action=none header.from=internalcorp.net
Content-Type: multipart/mixed; boundary="-----_Part_Boundary_7743992"

-----_Part_Boundary_7743992
Content-Type: text/html; charset="UTF-8"
Content-Transfer-Encoding: quoted-printable
```

Input

Y2h1Y2tfDgh1X2F0dGFjaG11bnQ=

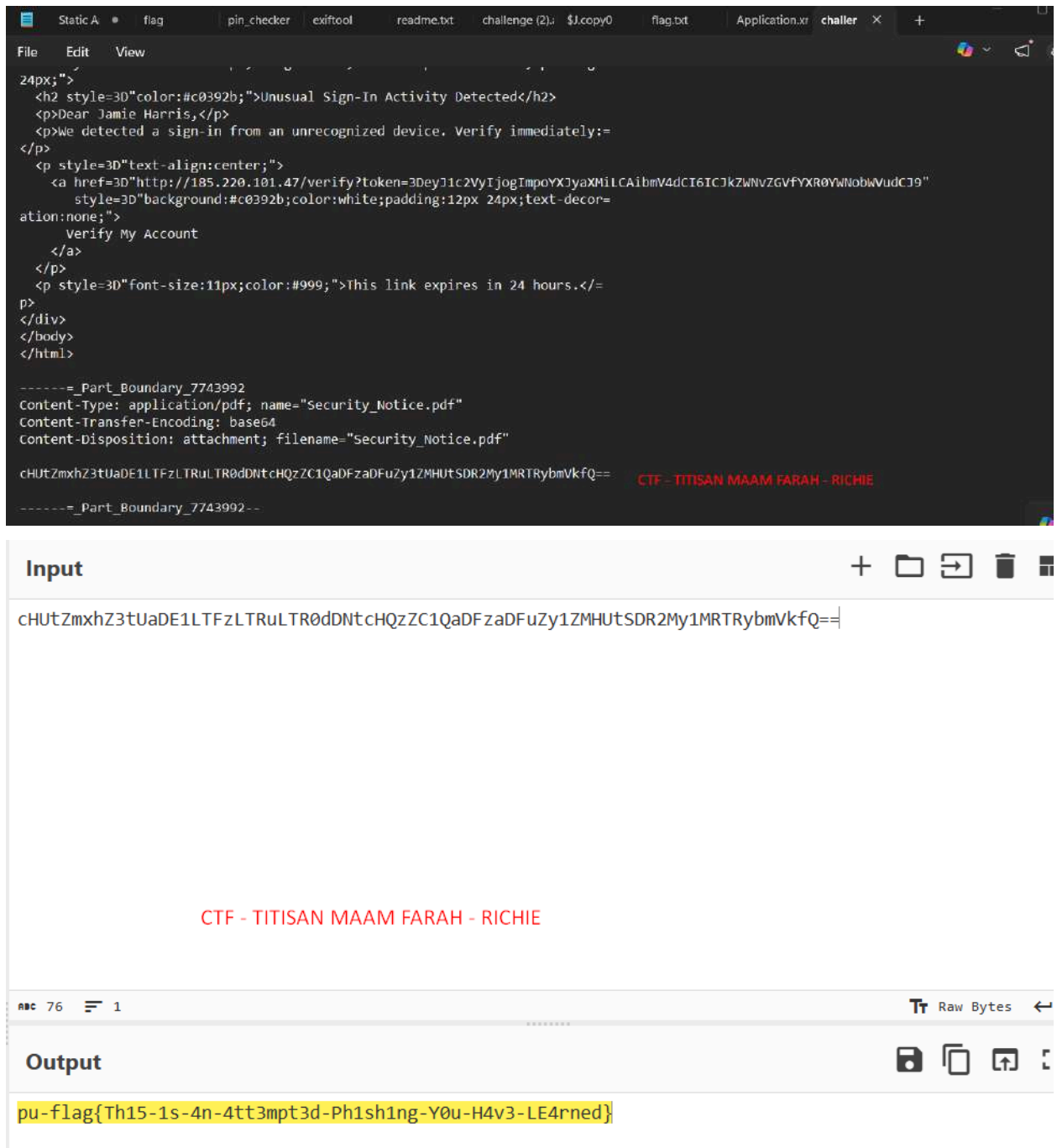
CTF - TITISAN MAAM FARAH - RICHIE

ABC 28 1

Output

check_the_attachment

6. After that, I saw the attachment pdf and it looks suspicious because like it has some base64 encode there, so i tried to input it in cyberchef again



The screenshot shows a web browser window with a security notice. The notice contains a link to verify the account. Below the notice is a PDF attachment named "Security_Notice.pdf". The PDF content is base64 encoded. The CyberChef interface shows the input string: `chUTZmxhZ3tUaDE1LTZlLTRuLTR0dDNTcHQzZC1QaDFzaDFuZy1ZMHUtSDR2My1MRTRYbmVkfQ==`. The output of the base64 decode operation is: `pu-flag{Th15-1s-4n-4tt3mpt3d-Ph1sh1ng-Y0u-H4v3-LE4rned}`.

```
Static A • flag pin_checker exiftool readme.txt challenge (2) $!copy0 flag.txt Application.xr challenger X +
File Edit View
24px;">
<h2 style=3D"color:#c0392b;">Unusual Sign-In Activity Detected</h2>
<p>Dear Jamie Harris,</p>
<p>We detected a sign-in from an unrecognized device. Verify immediately:=
</p>
<p style=3D"text-align:center;">
  <a href=3D"http://185.220.101.47/verify?token=3DeyJ1c2VyIjogImpoYXJyaXMlCAibmV4dCI6ICJkZWVhZGVfYXR0YWNobWVudCI9"
    style=3D"background:#c0392b;color:white;padding:12px 24px;text-decor=
ation:none;">
    Verify My Account
  </a>
</p>
<p style=3D"font-size:11px;color:#999;">This link expires in 24 hours.</=
p>
</div>
</body>
</html>

-----_Part_Boundary_7743992
Content-Type: application/pdf; name="Security_Notice.pdf"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="Security_Notice.pdf"

chUTZmxhZ3tUaDE1LTZlLTRuLTR0dDNTcHQzZC1QaDFzaDFuZy1ZMHUtSDR2My1MRTRYbmVkfQ== CTF - TITISAN MAAM FARAH - RICHIE
-----_Part_Boundary_7743992--

Input
+ [ ] [ ] [ ] [ ] [ ]

chUTZmxhZ3tUaDE1LTZlLTRuLTR0dDNTcHQzZC1QaDFzaDFuZy1ZMHUtSDR2My1MRTRYbmVkfQ==

CTF - TITISAN MAAM FARAH - RICHIE

abc 76 1 Tr Raw Bytes
Output
[ ] [ ] [ ] [ ] [ ]

pu-flag{Th15-1s-4n-4tt3mpt3d-Ph1sh1ng-Y0u-H4v3-LE4rned}
```

N then done, here's the flag.

scout-code2

Solved On: 13/4/2026

Solved by: Jesynta Ivolaitya Harya

Flag Retrieved: pu-flag{H4v3nt-Y0u-L34rn-M0rs3-cOd3-befOr3?}

Challenges overview:

This challenge required participants to decode a Morse code audio recording, use the resulting text as a steganography password, and extract a hidden flag from an image file. The challenge also contained a visual Morse code grid embedded in the image itself.

Key Findings:

- **Audio File — Morse Code**

The audio file morse_code.wav contained a continuous Morse code signal with no spacing between letters or words — one long unbroken beep sequence. This was intentional, requiring the song lyrics as a crib to reconstruct word boundaries.

- **Image File — Visual Morse Code Grid**

The file Flag.jpg contained a golden key graphic and three rows of black and white squares forming a visual Morse code grid. Each white square represented a dot or dash based on its width (1 cell = dot, 2 cells = dash).

- **Hidden Flag via Steganography**

Using the password extracted from the image morse grid with steghide on Flag.jpg successfully extracted the hidden flag:

pu-flag{H4v3nt-Y0u-L34rn-M0rs3-cOd3-befOr3?}

Forensic Analysis:

- **Audio Signal Analysis (Audacity)**

The audio file was opened in Audacity to visually inspect the waveform. The signal showed clearly separated blocks of constant tone, confirming it was a standard Morse code audio file. Each block was either a short pulse (dot) or a long pulse (dash).

Key measurements from the waveform:

- Dot duration: ~500ms
- Dash duration: ~1500ms
- Inter-symbol gap: ~100ms (uniform throughout)
- No letter or word gaps the entire sequence was continuous
- Total duration: 100 seconds

- **Morse Code Decoding (morsecode.world)**

The Morse code was manually transcribed from the Audacity waveform at 0.5x playback speed. The song hint (Still Here — League of Legends) was used to match word boundaries, as the audio contained no spacing between characters or words. The number 25 was identified at the end of the sequence. And get ITSINMYBLOODITSINMYVEINSIMSTILLHERE25

- **Image Analysis — Visual Morse Grid**

Flag.jpg was analysed for hidden content. The image contained three rows of black and white squares below a golden key graphic. Each row encoded Morse symbols:

- White squares spanning 2 grid cells = dash (-)
- White squares spanning 1 grid cell = dot (.)
- Black squares = gaps between symbols

The specific white boxes were selected from the grid and their positions read left to right across all three rows. The letters spelled out by the filled (white) squares gave the steganography password: ANTSINHEELS

- Steganography Extraction (steghide)

The steghide tool was used to extract hidden data from Flag.jpg using the password obtained from the visual morse grid.

```
steghide extract -sf Flag.jpg -p "antsinheels"
```

The extraction was successful and revealed the flag file containing:

pu-flag{H4v3nt-Y0u-L34rn-M0rs3-cOd3-befOr3?}

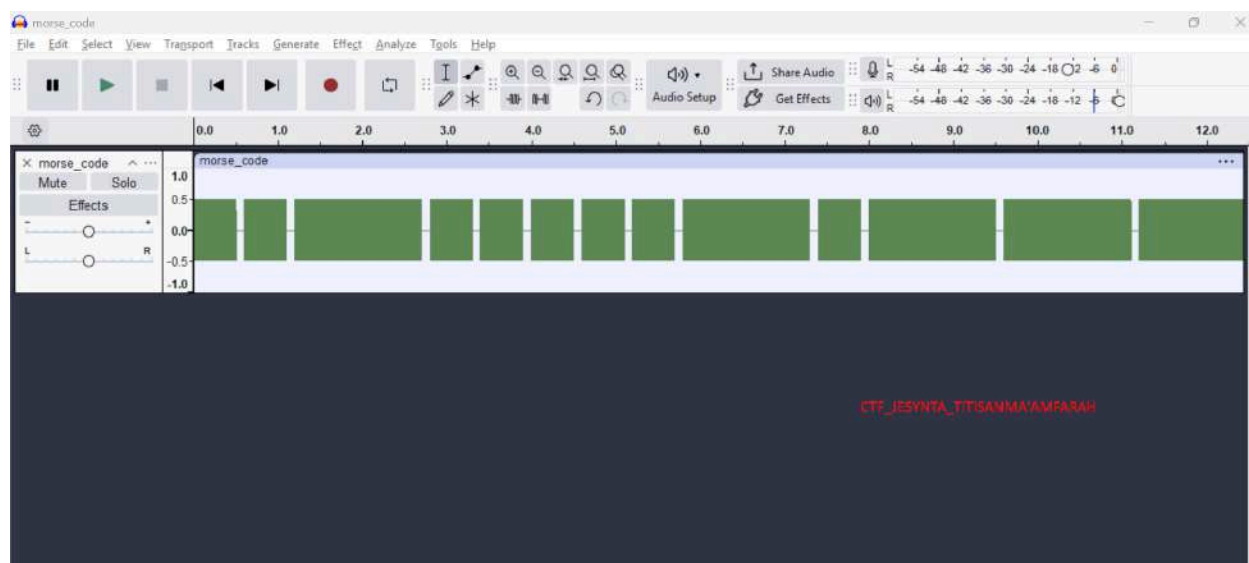
Tools Used:

- Audacity
- Morsecodeworld
- Steghide

Solving Step-by-step:

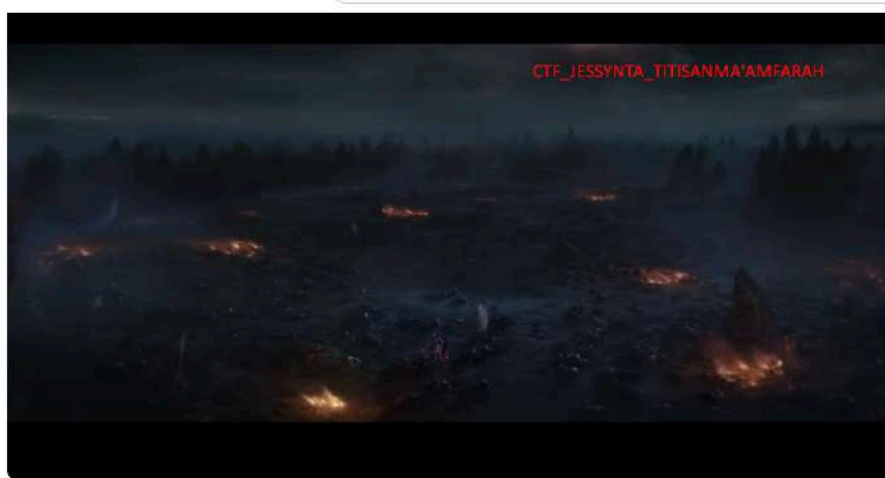
Step 1 — Open Audio in Audacity

Opened morse_code.wav in Audacity. Observed the waveform showed a continuous Morse signal alternating tones with no word/letter gaps. Played back at 0.5x speed to manually transcribe dots and dashes. And get .. - -. -- .-. ----. ---- --- -. ... - -. -- .-..



Step 2 — Identify the Song Hint

The moderator provided a YouTube link pointing to Still Here by League of Legends (2WEI, Forts, Tiffany Aris). The lyrics were used as a crib to reconstruct word boundaries in the Morse sequence since no gaps existed in the audio.



Still Here | Season 2024 Cinematic - League of Legends (ft. Forts, Tiffany Aris, and 2WEI)



League of Legends
16M subscribers

Subscribe

2M



Step 3 — Decode the Morse Code

.. - - , - - , - - - ... , .. - - - - - - - , - - , - - ... , .. - - ... - .. , .. , - ... - .. , .. , - ... - .. , .. , - ... - ...



Step 4 — Analyse Flag.jpg for Visual Morse

Reading the white boxes across the grid left to right:

Password = ANTSINHEELS



Step 5 — Extract Flag with Steghide

Using the decoded password from the visual morse grid in Flag.jpg, the steghide extract command was run in WSL:

```
jessy@Jessy:/mnt/d/CTF/Chall$ steghide extract -sf Flag.jpg -p "antsinheels"  
wrote extracted data to "flag.txt". CTF_JESYNTA_TITISANMA'AMFARAH  
jessy@Jessy:/mnt/d/CTF/Chall$
```

```
steghide extract -sf Flag.jpg -p "antsinheels"
```

The command successfully extracted the hidden file containing the flag.

Step 6 — Flag Obtained

Final Flag: pu-flag{H4v3nt-Y0u-L34rn-M0rs3-c0d3-bef0r3?}

